

Imbalanced Sample Sizes in RNA-seq Analyses

BIOS7659

Yifan Zhao

Table of Contents

<i>Introduction.....</i>	<i>1</i>
<i>Methods.....</i>	<i>2</i>
<i>Summary of Analysis Workflows.....</i>	<i>4</i>
<i>Results.....</i>	<i>6</i>
<i>Discussion and Conclusions.....</i>	<i>9</i>
<i>Code Appendix.....</i>	<i>10</i>
<i>Simulation.....</i>	<i>10</i>
<i>Helper Functions.....</i>	<i>11</i>
<i>Tests.....</i>	<i>15</i>

Introduction

In a previous project, we aimed to identify differentially expressed genes (DEGs) among three groups with roughly 10, 20, and 40 samples each using RNA-seq data. The largest group had nearly four times as many samples as the smallest, and the middle group had almost twice as many as the smallest. This substantial class imbalance raised questions about the validity and reliability of the differential expression analysis results. This issue of class imbalance is frequently discussed in bioinformatics forums. For instance, a Bioconductor user once asked whether it would be legitimate to conduct differential expression analysis on two groups with highly different sample sizes, such as 233 versus 6. The consensus was that, while technically feasible, the results might not be reliable due to the imbalance.

Motivated by the challenges encountered in our previous project, we decided to explore the effect of class imbalance on RNA-seq DEG finding procedures in more depth. Understanding how class imbalance impacts the accuracy and reliability of differential expression analysis is crucial for correctly understanding results from publications, improving analysis methodologies and ensuring robust results. In this project, we aim to systematically investigate the impact of class imbalance and develop guidelines for handling imbalanced datasets in RNA-seq DEG analysis.

Methods

To simulate the data, the SPsimSeq R package (1.10.0) was used, which can generate datasets based on real data. The source of the dataset to simulate is the demo dataset from this package, which contains 1000 genes and 100 samples, with 46 samples in one group and 54 samples in the other. We tested four sets of combinations between Group A and Group B samples to explore the impact of class imbalance on differential expression analysis. The combinations were designed to have varying degrees of imbalance to observe how the imbalance affects the results. Specifically, the combinations tested were 50 vs. 50, 40 vs. 60, 30 vs. 70, and 20 vs. 80 samples. For each setting, 200 datasets were simulated. A random seed of 7659 was set to ensure reproducibility.

Three mainstream R packages were chosen to analyze each dataset: edgeR (3.42.4), limma (3.56.2), and DESeq2 (1.40.2). A curated RNA-seq workflow combining variance stabilizing method and limma was also added to the comparison, which proves to be robust when the sample size is small. In total, each dataset was analyzed using 4 pipelines. Since each pipeline has its assumptions, the recommended setting was used to process the data. For example, in the

edgeR workflow, the Each pipeline contains the following key steps: setting up an object to operate with, filtering genes based on CPM or counts, transforming or normalizing data, model fitting and then testing hypotheses. A summary table with detailed code and notes can be found in Summary of Analysis Workflows.

To compare the results, number of DEGs being kept or filtered out, along with non-DEGs being kept or filtered out in each simulated dataset was recorded. Sensitivity, specificity, and AUC were calculated using pROC package (1.18.5) for each simulated dataset. The metrics mentioned above were then averaged across the 200 datasets within each condition. All analysis was performed in R 4.3.1. Code is available in the Appendix.

Summary of Analysis Workflows

Table 1 Summary of Major R code to Perform DEG Analysis for Each Pipeline

	edgeR	limma	DESeq2	Curated Workflow
Manual	https://www.bioconductor.org/packages/devel/bioc/vignettes/edgeR/inst/doc/edgeRUsersGuide.pdf	https://bioconductor.org/packages/release/bioc/vignettes/limma/inst/doc/usersguide.pdf	https://bioconductor.org/packages/devel/bioc/vignettes/DESeq2/inst/doc/DESeq2.html	
Design Matrix	design <- model.matrix(~ group)			
Object Setup	dge <- DGEList(counts=counts)		dds <- DESeqDataSetFromMatrix(countData = counts, colData = sample_info, design = ~ Group) min_grp_size <- min(table(sample_info\$Group))	
Filtering	keep <- filterByExpr(dge) dge <- dge[keep, , keep.lib.sizes = F]		keep <- rowSums(counts(dds) >= 10) >= min_grp_size dds <- dds[keep,]	cpm_mat # Calculate CPM keep <- rowSums(cpm_mat > 5) >= min_grp_size
Transformation And Normalization	dge <- calcNormFactors(dge)		dds <- DESeq(dds) result <- results(dds)	vsd <- varianceStabilizingTransformation(dds, blind = F, fitType = "parametric") expr_vsd <- assay(vsd)
	dge <- estimateDisp(dge, design)	logCPM <- cpm(dge, log=TRUE, prior.count=3)		
Model Fitting And Testing	fit <- glmQLFit(dge, design) qlf <- glmQLFTest(fit, coef = 2) result <- topTags(qlf)	fit <- lmFit(logCPM, design) fit <- eBayes(fit, trend=TRUE) result <- topTable(fit, coef=ncol(design))		fit_full <- lmFit(expr_vsd, design) fit_full <- eBayes(fit_full) result <- topTable(fit, coef=ncol(design))

Notes:

1. For the edgeR workflow:

- a. The quasi-likelihood method was applied since it is recommended by the authors because it gives stricter error rate by accounting for the uncertainty in dispersion estimation.
2. For the limma workflow:
 - a. It is advised that RNA-seq data should be scale normalized, such as by applying TMM method.
 - b. There are two approaches to analyze RNA-seq counts data using limma: limma-trend and voom. When the ratio of the largest library size to the smallest is not more than about 3-fold, it is recommended to use limma-trend approach.

For the consideration of consistency, the limma-trend approach was applied.
3. For the DESeq2 workflow:
 - a. The DESeq() function is a wrapper function to perform the three steps:
 - i. Estimation of size factors: estimateSizeFactors
 - ii. Estimation of dispersion: estimateDispersions
 - iii. Negative binomial GLM fitting and Wald statistics: nbinomWaldTest
4. For the curated workflow:
 - a. The object was created after the filtering step using the filtered count matrix.
 - b. Although CPM calculation is also a transformation of the data, the purpose of utilizing CPM is to filter genes.

Results

In order to compare the results, we first look at the average number of DEGs and non-DEGs being kept or removed in each setting. From Table 2, the first two workflows, edgeR and limma always report the same numbers since they share the same gene-filtering workflow. The general trend remain the same across different conditions, as shown in Figure 1. However, it is worth-noting that despite the workflow, a great fraction of DEGs, roughly 2/3, were removed because of the initial filtering step. However, as the degree of imbalance increases, the number of genes being filtered out decreases. This is because most method keeps genes based on the smallest number of samples in the groups.

Table 2 Summary Table of DEGs/non-DEGs Being Filtered/Kept in Each Condition

		Filtered non-DEGs	Kept non-DEGs	Filtered DEGs	Kept DEGs
50-50	edgeR	1030	1670	204	96
	limma	1030	1670	204	96
	DESeq2	1091	1609	228	72
	Curated Workflow	1070	1630	223	77
40-60	edgeR	996	1704	192	108
	limma	996	1704	192	108
	DESeq2	1042	1658	207	93
	Curated Workflow	1019	1681	203	97
30-70	edgeR	956	1744	179	121
	limma	956	1744	179	121
	DESeq2	990	1710	192	108
	Curated Workflow	966	1734	185	115
20-80	edgeR	901	1799	165	135
	limma	901	1799	165	135
	DESeq2	926	1774	171	129
	Curated Workflow	901	1799	166	134

We further compared the sensitivity, specificity, and AUC of each method under each condition, as shown in Table 3 and Figure 2. As shown in Figure 2, the metrics change

slightly when the degree of imbalance changes. The curated workflow performs best in all situations, while edgeR tends to have the worst performance in all situations.

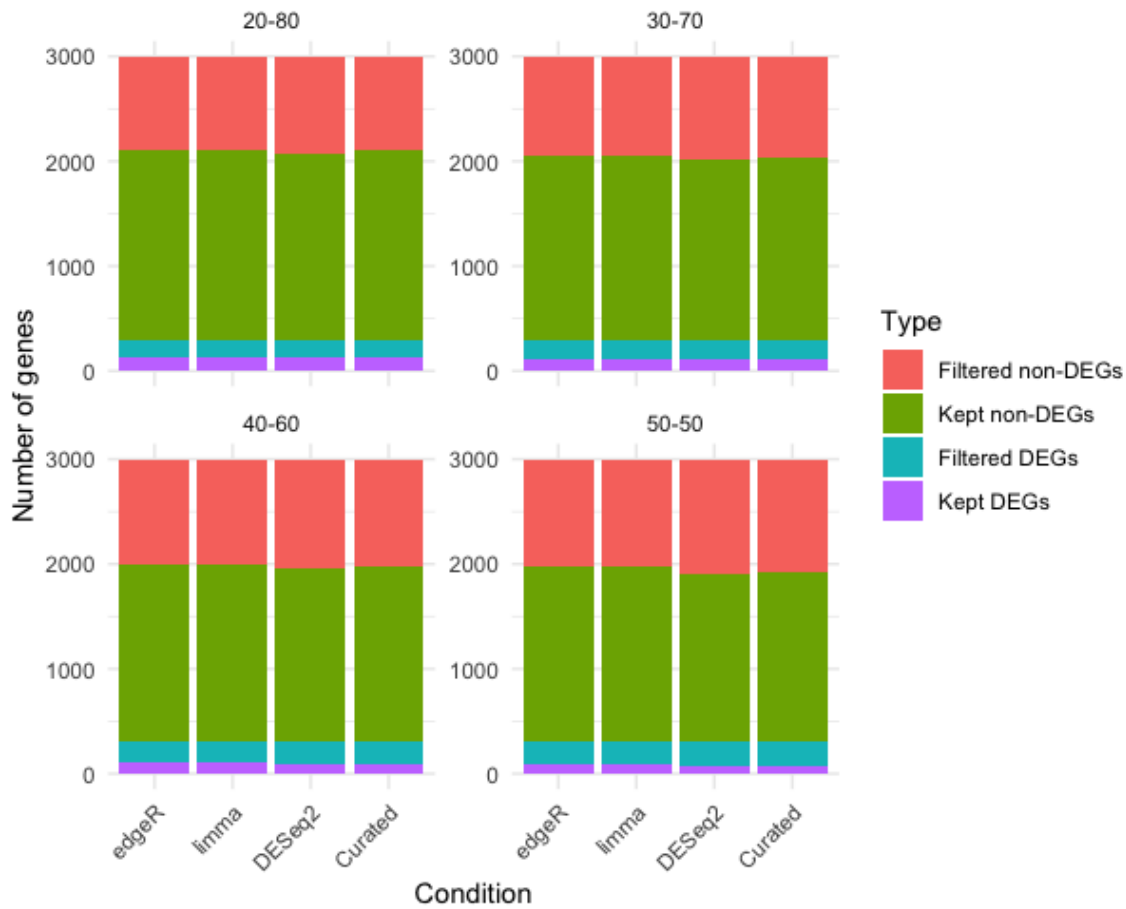


Figure 1 Bar Plot of DEGs/non-DEGs Being Kept/Filtered

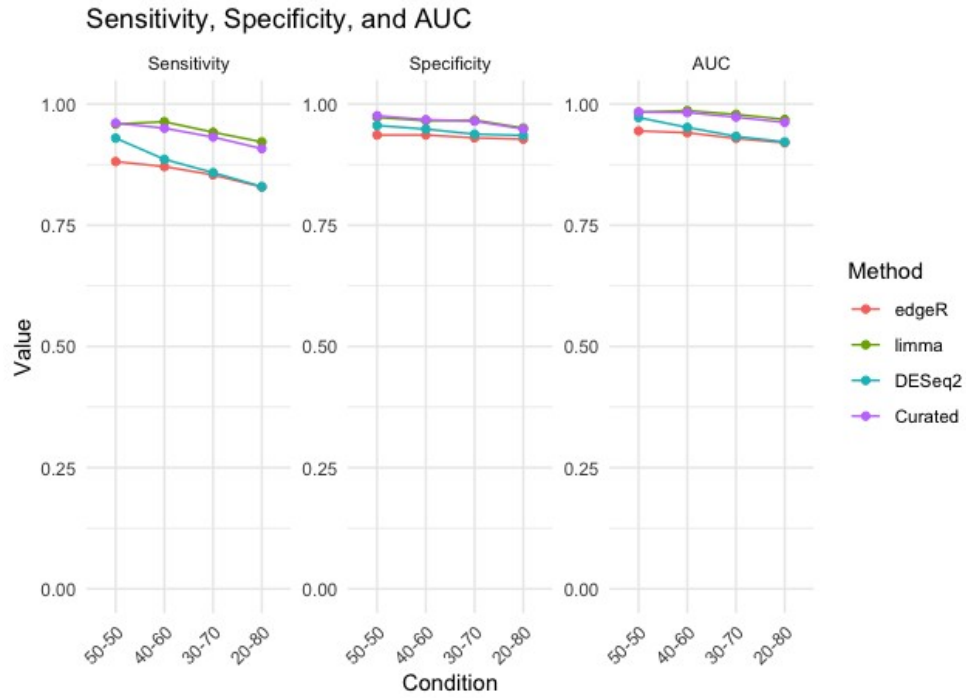


Figure 2 Sensitivity, Specificity, and AUC Based on Degrees of Imbalance

Table 3 Summary of Sensitivity, Specificity, and AUC

		Sensitivity	Specificity	AUC
50-50	edgeR	0.881	0.936	0.944
	limma	0.959	0.972	0.983
	DESeq2	0.930	0.956	0.972
	Curated Workflow	0.961	0.976	0.984
40-60	edgeR	0.871	0.936	0.941
	limma	0.963	0.965	0.986
	DESeq2	0.886	0.948	0.952
	Curated Workflow	0.950	0.968	0.982
30-70	edgeR	0.854	0.930	0.929
	limma	0.942	0.967	0.979
	DESeq2	0.858	0.938	0.933
	Curated Workflow	0.932	0.965	0.973
20-80	edgeR	0.829	0.927	0.920
	limma	0.922	0.950	0.968
	DESeq2	0.829	0.935	0.922
	Curated Workflow	0.908	0.948	0.962

Discussion and Conclusions

The study focused on the impact of class imbalance in sample sizes on RNA-seq differential gene expression analysis, using four different analytic pipelines. The results indicated that varying degrees of imbalance did indeed influence the detection of differentially expressed genes (DEGs). Among the methods compared, the curated workflow consistently performed well across all conditions in terms of sensitivity, specificity, and AUC, suggesting its robustness against the effects of class imbalance. The study highlights the importance of considering sample size balance in RNA-seq analyses to ensure accurate and reliable identification of DEGs, providing insights that could help improve methodologies for handling imbalanced datasets in genomic research.

However, the result is limited in the following ways. First, it would be more meaningful to control the number of samples in one group and then increase the sample size of the other, so that the minimum group size remains the same, and we would be able to see the trend of gene being filtered out based on degree of imbalance. This would also allow us to vary the total number of samples. Second, important steps such as outlier detection, normalization, batch effects were not considered in this study, which is usually not the best practice in real life. Further tests should be performed to gain a better understanding of the impact of class imbalance in the study design.

Code Appendix

Simulation

```
library(SPsimSeq)

# Example -----
# load the Zhang bulk RNA-seq data (availabl with the package)
data("zhang.data.sub")

# filter genes with sufficient expression (important step to avoid bugs)
zhang.counts <- zhang.data.sub$counts
MYCN.status <- zhang.data.sub$MYCN.status #The grouping variable

dim(zhang.counts)
# 1000 100

length(MYCN.status)
# 100
table(MYCN.status)
# MYCN.status
# 0 1
# 46 54

set.seed(6452) #Set seed for reproducibility
# simulate data
sim.data.bulk <- SPsimSeq(
  n.sim = 1, s.data = zhang.counts,
  group = MYCN.status, n.genes = 3000, batch.config = 1,
  group.config = c(0.5, 0.5), tot.samples = ncol(zhang.counts),
  pDE = 0.1, lfc.thrld = 0.5,
  result.format = "list", return.details = TRUE)

set.seed(7659)
sim.data.bulk <- SPsimSeq(
  n.sim = 200, s.data = zhang.counts,
  group = MYCN.status, n.genes = 3000, batch.config = 1,
  group.config = c(0.5, 0.5), tot.samples = ncol(zhang.counts),
  pDE = 0.1, lfc.thrld = 0.5,
  result.format = "list", return.details = TRUE)

# Save the data
saveRDS(sim.data.bulk, "DataProcessed/sim.data.bulk_5_5.rds")

sim.data.bulk <- SPsimSeq(
  n.sim = 200, s.data = zhang.counts,
  group = MYCN.status, n.genes = 3000, batch.config = 1,
  group.config = c(0.4, 0.6), tot.samples = ncol(zhang.counts),
  pDE = 0.1, lfc.thrld = 0.5,
  result.format = "list", return.details = TRUE)

# Save the data
saveRDS(sim.data.bulk, "DataProcessed/sim.data.bulk_4_6.rds")
```

```

sim.data.bulk <- SPsimSeq(
  n.sim = 200, s.data = zhang.counts,
  group = MYCN.status, n.genes = 3000, batch.config = 1,
  group.config = c(0.3, 0.7), tot.samples = ncol(zhang.counts),
  pDE = 0.1, lfc.thrld = 0.5,
  result.format = "list", return.details = TRUE)
# Save the data
saveRDS(sim.data.bulk, "DataProcessed/sim.data.bulk_3_7.rds")

sim.data.bulk <- SPsimSeq(
  n.sim = 200, s.data = zhang.counts,
  group = MYCN.status, n.genes = 3000, batch.config = 1,
  group.config = c(0.2, 0.8), tot.samples = ncol(zhang.counts),
  pDE = 0.1, lfc.thrld = 0.5,
  result.format = "list", return.details = TRUE)
# Save the data
saveRDS(sim.data.bulk, "DataProcessed/sim.data.bulk_2_8.rds")

```

Helper Functions

Helper functions

```

# Function to calculate the sensitivity, specificity, and auc
calc_sens_spec_auc <- function(gene_info, result, method) {
  merged_data <- gene_info %>%
    rownames_to_column("gene") %>%
    left_join(result %>% as.data.frame() %>%
      rownames_to_column("gene"), by = "gene")
  if (method == "edgeR") {
    roc_curve <- roc(merged_data$DE.ind, merged_data$FDR)
  } else if (method == "DESeq2") {
    roc_curve <- roc(merged_data$DE.ind, merged_data$padj)
  } else { # limma and Brian
    roc_curve <- roc(merged_data$DE.ind, merged_data$adj.P.Val)
  }

  sens <- coords(roc_curve, "best", ret = "sensitivity")
  spec <- coords(roc_curve, "best", ret = "specificity")

  auc_score <- auc(roc_curve)
  return(c(sens, spec, auc_score))
}

# Function to calculate the counts of filtered genes
calc_filt_genes <- function(gene_info, keep_vec) {
  gene_info$keep <- keep_vec
  comb_counts <- table(gene_info$keep, gene_info$DE.ind) %>%
    as.data.frame()
  comb_counts <- comb_counts[,3]
  names(comb_counts) <- c("filtered_EE", "kept_EE",
    "filtered_DE", "kept_DE")
  return(comb_counts)
}

```

```

# Function to run edgeR
run_edgeR <- function(counts, sample_info, gene_info) {
  group <- sample_info$Group
  dge <- DGEList(counts = counts, group = group)
  keep <- filterByExpr(dge)
  combo_counts <- calc_filt_genes(gene_info, keep)
  dge <- dge[keep, , keep.lib.sizes = F]
  dge <- calcNormFactors(dge)
  design <- model.matrix(~ group)
  dge <- estimateDisp(dge, design)
  fit <- glmQLFit(dge, design)
  qlf <- glmQLFTest(fit, coef = 2)
  edgeR_result <- topTags(qlf, n = Inf, p.value = 1)
  return(list(keep_counts = combo_counts, tbl = edgeR_result))
}

# Function to run limma
run_limma <- function(counts, sample_info, gene_info) {
  group <- sample_info$Group
  dge_limma <- DGEList(counts = counts, group = group)
  keep_limma <- filterByExpr(dge_limma)
  combo_counts <- calc_filt_genes(gene_info, keep_limma)
  dge_limma <- dge_limma[keep_limma, , keep.lib.sizes = F]
  dge_limma <- calcNormFactors(dge_limma)
  logCPM <- cpm(dge_limma, log = T, prior.count = 3)
  design_limma <- model.matrix(~ group)
  fit_limma <- lmFit(logCPM, design_limma)
  fit_limma <- eBayes(fit_limma, trend = T)
  limma_result <- topTable(fit_limma, coef = 2, n = Inf, p.value = 1)
  return(list(keep_counts = combo_counts, tbl = limma_result))
}

# Function to run DESeq2
run_DESeq2 <- function(counts, sample_info, gene_info) {
  dds <- DESeqDataSetFromMatrix(countData = counts,
                                colData = sample_info,
                                design = ~ Group)
  smallestGroupSize <- min(table(sample_info$Group))
  keep <- rowSums(counts(dds) >= 10) >= smallestGroupSize
  combo_counts <- calc_filt_genes(gene_info, keep)
  dds <- dds[keep,]

  dds <- DESeq(dds)
  res <- results(dds)

  return(list(keep_counts = combo_counts, tbl = res))
}

# Function to run Brian's method
run_Brian <- function(counts, sample_info, gene_info) {
  min_grp_size <- min(table(sample_info$Group))

  cpm_mat <- apply(counts, 2, function(xx){
    return(xx / sum(xx) * 1e6)
  })

```

```

idx_keep <- rowSums(cpm_mat > 5) >= min_grp_size

combo_counts <- calc_filt_genes(gene_info, idx_keep)

# Filter raw counts, CPM, and gene_names objects
counts_only_filt <- counts[idx_keep, ]
cpm_mat_filt <- cpm_mat[idx_keep, ]

# VST
dds <- DESeqDataSetFromMatrix(countData = counts_only_filt,
                              colData = sample_info,
                              design = ~ Group)
vsd <- varianceStabilizingTransformation(dds, blind = F,
                                          fitType = "parametric")
expr_vsd <- assay(vsd)

# limma
# Generate regression model matrix
dmat <- model.matrix(~ sample_info$Group)

# Get initial regression modelling results
fit_full <- lmFit(object = expr_vsd, design = dmat)

# Apply the empirical Bayes procedure
fit_full <- eBayes(fit_full)

# Get estimated log scale fold changes and adjusted p-values for the difference between PBS and DEP mice
tt_exposure <- topTable(fit_full, number = Inf, p.value = 1, sort.by = "none")

return(list(keep_counts = combo_counts, tbl = tt_exposure))
}

# Function to run the DE analysis given the simulated datasets
run_DE_analysis <- function(rds_file) {
  # Load the data
  sim.data <- readRDS(rds_file)

  ndatasets <- length(sim.data$sim.data.list)

  # Initialize a data frame with 200 rows and 16 columns
  combo_counts <- data.frame(matrix(ncol = 16, nrow = ndatasets))
  colnames(combo_counts) <- c(
    "filtered_EE_edgeR", "kept_EE_edgeR",
    "filtered_DE_edgeR", "kept_DE_edgeR",
    "filtered_EE_limma", "kept_EE_limma",
    "filtered_DE_limma", "kept_DE_limma",
    "filtered_EE_DESeq2", "kept_EE_DESeq2",
    "filtered_DE_DESeq2", "kept_DE_DESeq2",
    "filtered_EE_Brian", "kept_EE_Brian",
    "filtered_DE_Brian", "kept_DE_Brian")

  # Initialize a data frame with 200 rows and 12 columns
  sens_spec <- data.frame(matrix(ncol = 12, nrow = ndatasets))
  colnames(sens_spec) <- c(
    "edgeR_sensitivity", "edgeR_specificity", "edgeR_auc",
    "limma_sensitivity", "limma_specificity", "limma_auc",

```

```

"DESeq2_sensitivity", "DESeq2_specificity", "DESeq2_auc",
"Brian_sensitivity", "Brian_specificity", "Brian_auc")

for (i in 1:ndatasets) {
  dataset <- sim.data$sim.data.list[[i]]
  cnt <- dataset$counts
  smp <- dataset$colData
  gene_info <- dataset$rowData

  edgeR_result <- run_edgeR(cnt, smp, gene_info)
  limma_result <- run_limma(cnt, smp, gene_info)
  DESeq2_result <- run_DESeq2(cnt, smp, gene_info)
  Brian_result <- run_Brian(cnt, smp, gene_info)

  # Store the number of genes in the dataset
  combo_counts[i, 1:4] <- edgeR_result$keep_counts
  combo_counts[i, 5:8] <- limma_result$keep_counts
  combo_counts[i, 9:12] <- DESeq2_result$keep_counts
  combo_counts[i, 13:16] <- Brian_result$keep_counts

  # Calculate sensitivity, specificity and AUC
  sens_spec[i, 1:3] <- calc_sens_spec_auc(gene_info,
                                          edgeR_result$tbl,
                                          "edgeR")
  sens_spec[i, 4:6] <- calc_sens_spec_auc(gene_info,
                                          limma_result$tbl,
                                          "limma")
  sens_spec[i, 7:9] <- calc_sens_spec_auc(gene_info,
                                          DESeq2_result$tbl,
                                          "DESeq2")
  sens_spec[i, 10:12] <- calc_sens_spec_auc(gene_info,
                                             Brian_result$tbl,
                                             "Brian")
}

return(list(combo_counts, sens_spec))
}

```

Tests

```

library(dplyr)
library(tidyr)
library(pROC)

# Test over sim.data.bulk_5_5 -----
results_5_5 <- run_DE_analysis("DataProcessed/sim.data.bulk_5_5.rds")

results_5_5_kept <- results_5_5[[1]]
results_5_5_sens_spec <- results_5_5[[2]]

# Test over sim.data.bulk_4_6 -----
results_4_6 <- run_DE_analysis("DataProcessed/sim.data.bulk_4_6.rds")

results_4_6_kept <- results_4_6[[1]]
results_4_6_sens_spec <- results_4_6[[2]]

```

```

# Test over sim.data.bulk_3_7 -----
results_3_7 <- run_DE_analysis("DataProcessed/sim.data.bulk_3_7.rds")

results_3_7_kept <- results_3_7[[1]]
results_3_7_sens_spec <- results_3_7[[2]]

# Test over sim.data.bulk_2_8 -----
results_2_8 <- run_DE_analysis("DataProcessed/sim.data.bulk_2_8.rds")

results_2_8_kept <- results_2_8[[1]]
results_2_8_sens_spec <- results_2_8[[2]]

# Summary -----
results_kept <- rbind(colMeans(results_5_5_kept),
                     colMeans(results_4_6_kept),
                     colMeans(results_3_7_kept),
                     colMeans(results_2_8_kept))
write.csv(results_kept, "DataProcessed/0515_results_kept.csv")

results_sens_spec <- rbind(colMeans(results_5_5_sens_spec),
                          colMeans(results_4_6_sens_spec),
                          colMeans(results_3_7_sens_spec),
                          colMeans(results_2_8_sens_spec))
write.csv(results_sens_spec, "DataProcessed/0515_results_sens_spec.csv")

# Plotting -----
View(results_kept)
results_kept_long <- results_kept %>% as.data.frame() %>%
  mutate(Condition = c("50-50", "40-60", "30-70", "20-80")) %>%
  pivot_longer(cols = -Condition, names_to = "Category",
               values_to = "Value")
# Split the column Category into two columns: Type and Method based on the last "_"
results_kept_long <- results_kept_long %>%
  mutate(Type = sub("^[^_]*$", "", Category),
         Method = sub(".*_", "", Category))
results_kept_long <- results_kept_long %>%
  mutate(Method = factor(Method, levels = c("edgeR", "limma",
                                           "DESeq2", "Brian"),
                        labels = c("edgeR", "limma", "DESeq2", "Curated"))) %>%
  mutate(Type = factor(Type, levels = c("filtered_EE", "kept_EE",
                                       "filtered_DE", "kept_DE"),
                      labels = c("Filtered non-DEGs", "Kept non-DEGs",
                                "Filtered DEGs", "Kept DEGs")))

# Make a stacked bar plot: stratified by condition, a bar for each method
ggplot(results_kept_long, aes(x = Method, y = Value, fill = Type)) +
  geom_bar(stat = "identity", position = "stack") +
  facet_wrap(~Condition, scales = "free_y") +
  theme_minimal() +
  labs(title = "Number of genes kept after filtering",
       x = "Condition",
       y = "Number of genes") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

```

# Line chart using results_sens_spec -----
results_sens_spec_long <- results_sens_spec %>% as.data.frame() %>%
  mutate(Condition = c("50-50", "40-60", "30-70", "20-80")) %>%
  pivot_longer(cols = -Condition, names_to = "Category",
    values_to = "Value")
# Split the column Category into two columns: Type and Method based on the last "_"
results_sens_spec_long <- results_sens_spec_long %>%
  mutate(Method = sub("_[^_]*$", "", Category),
    Metric = sub(".*_", "", Category))
results_sens_spec_long <- results_sens_spec_long %>%
  mutate(Method = factor(Method, levels = c("edgeR", "limma",
    "DESeq2", "Brian"),
    labels = c("edgeR", "limma", "DESeq2", "Curated"))) %>%
  mutate(Metric = factor(Metric, levels = c("sensitivity", "specificity", "auc"),
    labels = c("Sensitivity", "Specificity", "AUC")))

results_sens_spec_long$Condition <- factor(results_sens_spec_long$Condition, levels = c("50-50", "40-60", "30-70",
"20-80"))

# Make a line chart: stratified by condition, a line for each metric
ggplot(results_sens_spec_long, aes(x = Condition, y = Value, color = Method, group = Method)) +
  geom_line() +
  geom_point() +
  facet_wrap(~Metric, scales = "free_y") +
  theme_minimal() +
  labs(title = "Sensitivity, Specificity, and AUC",
    x = "Condition",
    y = "Value") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  ylim(0, 1)

```